Experiment - 6

Aim - To write a program to implement error detection and correction using Hamming code concept, and make a test run.

Code:

```
def redundant_bits(m):
    r = 0
    while 2** r < m + r + 1:
        r += 1
    return r

def encode(data):
    m = len(data)
    r = redundant_bits(m)
    print("Number of data bits:", m)
    n = m + r
    h = ['0'] * (n+1)
    j = 0
```

```
for p in range(1, r+1):
    p = 2**p
    parity = 0
    for j in range(1, n+1):
        if j & p:
            parity ^= int(h[j])
    h[p] = str(parity)
return ''.join(h[1:])

def check(h):
    n = len(h)
    r = 0
    while 2**r <= n:
        r += 1
    error = 0
    print("In Parity Checking:")
    for i in range(r):
        p = 2**i
        parity = 0
```

```
def remove_parity(h):
    data = ""
    for i in range(1, len(h)+1):
        if(i%8!=0):
            data += h[i-1]
    return data

def binary_to_text(data):
    text = ''
    for i in range(0, len(data), 8):
        byte = data[i:i+8]
        text += chr(int(byte,2))
    return text.

change = input("\n Do you want to change the
transmitted data? (yes/no): ")
if change.lower()=="yes":
    pos = int(input("Enter position to change: "))
```

```
if h[pos-1] == '0':
    h[pos-1] = '1'

else:
    h[pos-1] = '0'

received = ''.join(h)
print("Changed Data :")
print(received)

else:
    print("Invalid Position")

hamming = list(received)
hamming = check(hamming)
data = remove_parity(hamming)
print("Data after removing:")
print("\n Output:", output)
```

I/P

Enter data you want to transmit : 41.

Yes.

O/P

data : 41

Binary : 0100 1000 1001 1001

Hamming code : 1011 0011 0000 1001 1001

received data:

1011 0011 0000 0101 1001

Result : Thus the program is implemented using Python for error correction and detection.